

TeraiNet: An open-source blueprint for the fast and flexible development of local species classification models

Alexander V. Merdian-Tarko^{1*}

¹ Analytics Department, REWE Group, Domstraße 20, 50668 Köln, Germany

* alexander.merdian-tarko@posteo.de

Abstract: Automatically classifying wildlife in camera trap images using machine learning (ML) has the potential to boost conservation efforts. Developing and deploying such ML models can be challenging though. Therefore, an open-source blueprint is proposed for the fast and flexible development of local species classification models with a focus on accessibility, reproducibility, and cost-efficiency. The blueprint is put into practice through TeraiNet – a novel species classification model to support Bengal tiger conservation in the Terai Arc Landscape in Nepal. This region is a vital biodiversity hotspot, home to several endangered species. TeraiNet is built on EfficientNetV2, trained with open data (e.g., LILA BC), utilises open-source tools (e.g., MegaDetector, MEWC), and is openly available in AddaxAI for use by conservationists. Freely accessible platforms like Google Colab and Kaggle provide the compute environment for model training and evaluation. Despite compromises in data (e.g., limited local camera trap images and substitution with non-local species) and constraints in compute capacity, the model achieves promising performance. Through the example of TeraiNet, the blueprint aims to enable ecologists, NGOs, government agencies, and citizen scientists to build their own local species classifiers for their own use cases.

1. Introduction

Applications of artificial intelligence (AI) and machine learning (ML) are increasingly being implemented to support conservation efforts [1]. Camera traps, for example, are widely used, effective tools for different conservation efforts, such as monitoring wildlife populations. Such camera traps can collect large amounts of data (e.g., millions of images) in a single survey. Processing these amounts of data manually constitutes a bottleneck as time is often a limiting factor for conservationists. Therefore, automatically processing images from camera trap surveys using ML poses a great opportunity to speed up the entire process, freeing resources for other important conservation tasks [2].

It can be challenging, however, to fully exploit the potential of ML for conservation due to a lack of suitable (high-quality) data, ML skills, and compute resources. A range of tools is available to bridge parts of this gap and lower the barrier to entry regarding different aspects, including amongst others AddaxAI [3], DeepFaune [4], MegaDetector (MD) [5], Mega Efficient Wildlife Classifier (MEWC) [6], WildlifeInsights [7], and Zamba [8].

Consequently, to further bridge this gap, this work proposes an open-source blueprint for the development of local species classification models to support conservation. The blueprint is designed to enable conservationists, ecologists, NGOs, government agencies, and citizen scientists to build their own species classifiers for their own local use cases in a fast and flexible manner – either as a stand-alone solution or as the basis for other downstream tasks, such as population estimation. In its initial proposed form, it relies on openly available images (e.g., LILA BC [9]), open tools (e.g., AddaxAI, MD, MEWC), and utilises freely accessible, but restricted storage and compute resources (e.g., Google Drive [10], Google Colab [11], and Kaggle [12]). It is worth noting that MEWC is Docker-

based and therefore can't be used directly with Colab or Kaggle since both lack GPU-ready Docker compatibility. The blueprint is especially effective if users don't have their own GPUs and are interested in only a few/single species for which a sufficient number of images are openly available. At its core, the blueprint follows the approach of applying a detector step to generically detect animals in images using MD and a subsequent classification step where the trained model classifies the previously detected animals into a set of defined classes [13]. The proposed blueprint is designed with openness, reproducibility, and ease of use in mind, relying on established tools and trying to reduce large parts of the hassle of setting up an entire ML pipeline end-to-end or invoking prohibitive costs, thus striking a balance between effectiveness and cost-efficiency.

To demonstrate how it is supposed to work and to address an actual conservation challenge, the blueprint is put into practice by developing and deploying a species classification model – *TeraiNet* – with the ambition to support Bengal tiger conservation in the Terai Arc Landscape in southern Nepal [14]. The latter is not only home to over six million people but is also an important ecosystem, housing a variety of emblematic, partly endangered, species including the Bengal tiger, the Asian elephant and the Indian rhinoceros [15]. Great efforts have been thus undertaken to restore and conserve this biodiversity hotspot for humans and wildlife alike, and significant progress has been made in that direction [16]. While there remain limitations and challenges, concerning both the blueprint and the model itself, the preliminary results of TeraiNet appear promising.

Figure 1 shows a flow chart of the individual steps constituting the blueprint. The remainder of this work explains these steps, using the example of TeraiNet where applicable, including presenting the data used, the model design, experimental results, limitations and challenges, and finally a conclusion with a summary and outlook.

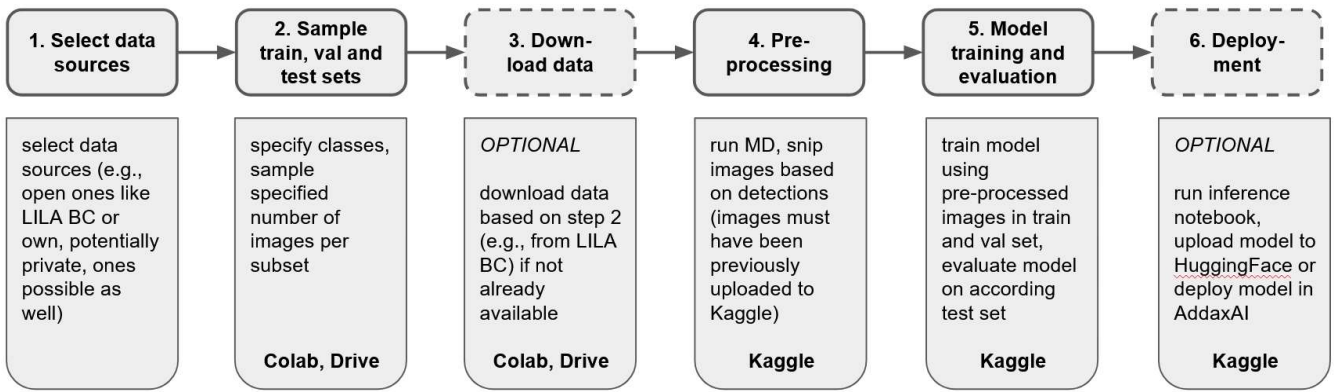


Fig. 1. Flow chart of the individual steps constituting the blueprint, including the respective tools involved

2. Data

This section describes the image sources used for TeraiNet, how images are sampled and obtained, and how they are further pre-processed for model training. See Table 2 in the Appendix for details on the number of images per selected development pipeline step of TeraiNet.

2.1. Sources

Images for TeraiNet stem mostly from LILA BC, a public archive of several camera trap datasets, covering a wide range of species and ecosystems. Although there is a great abundance of images and species on LILA BC there are only few images of tigers available there (~300), which constitute TeraiNet’s primary target species though. Furthermore, there seem to be no other sources for openly available camera trap images of tigers in general, and Bengal tigers specifically. One potential reason for this might be that tigers are an endangered species and therefore prone to risks, such as poaching. Therefore, to have a tiger class with more than a couple hundred images, additional images are taken from the Amur tiger re-identification challenge at CVWC 2019 [17] (*tiger substitute dataset*), which comprises images of Amur tigers taken in Chinese zoos. These are not an ideal substitute because they are significantly different from actual camera trap images taken in Nepal. Thus, special attention needs to be directed to how well the model trained on such imperfect substitute images generalises to actual local camera trap images of tigers. In a first attempt to investigate TeraiNet’s capacity to generalise to actual camera trap images of tigers, the tiger camera trap images from LILA BC are withheld in an additional test set.

Although the proposed blueprint currently focuses on openly available images on LILA BC, users can include their own, potentially private, images as well (e.g., analogous to the tiger substitute dataset).

2.2. Sampling

On LILA BC, there are no camera trap images available for the Terai region specifically, which already constitutes a challenge in and of itself. Besides, there are no camera trap images openly available for specific species that are present in Nepal. Therefore, a random sample is taken for each of the following ten classes that have been derived

as the relevant ones for this ecosystem (focus on mammals and birds), including substitutes and enrichments where there are either no or not enough images available for the actual target species (see Table 1). The classes *black bear* and *elephant* contain additional similar species for enrichment while the classes *buffalo* and *rhinoceros* contain substitute species only (i.e., African and Cape buffalo for gaur; White rhinoceros for Indian rhinoceros). The class *other carnivores* contains both species for enrichment and for substitution (e.g., black-backed jackal and gray fox).

This constitutes a significant compromise with the objective of having at least images of species that are similar rather than none. It remains to be seen how well this strategy works when the model is applied to local camera trap images of the actual species in question.

Table 1 Class and species assignment

Class	Species
(1) Tiger	Tiger
(2) Leopard	Leopard
(3) Black bear	Asian & American black bear
(4) Other carnivores	Dhole, black-backed jackal, gray fox, leopard cat, mainland leopard cat, marbled cat, Asian golden cat
(5) Deer	Deer
(6) Wild boar	Wild boar
(7) Buffalo	African and Cape buffalo
(8) Rhinoceros	White rhinoceros
(9) Elephant	Asian & African bush elephant
(10) Bird	Bird

All ~4,700 tiger images are taken from the tiger substitute dataset (tiger images from LILA BC are withheld from this step to investigate generalisation capacity), and ~3,000 images are randomly sampled for each of the other classes. The images are randomly split into training, validation and test subsets, considering location to avoid data leakage (i.e., per class, no location appears in more than one subset). The number of images is approximately distributed according to the ratio 80:10:10. Different locations have varying numbers of images and so the distribution, under the location constraint, is not always

exact regarding the desired ratio. This initially results in slightly unequal numbers of images per subset across classes which is not an issue though because the images are sampled again right before training, finally yielding equal subset sizes across classes.

The sampled images (URLs stored in CSV files for reproducibility) are downloaded from LILA BC using a combination of a Colab notebook and a Bash script, and they are temporarily stored in Drive. The downloaded images are copied into a joint directory, compressed, and uploaded to Kaggle as a (public or private) dataset for further pre-processing.

2.3. Pre-processing

The images contained in the Kaggle dataset are processed by MD (v5a) in a (public or private) Kaggle notebook, using two NVIDIA Tesla T4 GPUs. A rather conservative detection threshold of 0.6 is used to increase the likelihood of an actual animal being detected in an image in contrast to a random artifact, maximising down-the-line image quality for training. This threshold could be tuned more thoroughly in a future work. The resulting detections are cropped using functionality from MEWC, and the cropped images are saved as an output of the respective Kaggle notebook.

3. Model design

The model design is oriented at the MEWC implementation but simplified to better comply with computational restrictions. These simplifications mainly concern the input size and the base model’s classification head. In a next iteration, using a more complex model head and a larger image size could be tested to improve predictive performance. EfficientNetV2 [18] (pre-trained) is used as the base architecture. Priority is given to using well-established tools (e.g., MD and EfficientNetV2) over model innovation to reduce the blueprint’s complexity and increase its ease of use. The input size is fixed at 224x224 pixels although larger variants of the base architecture can work with larger input sizes, enabling the recognition of more fine-grained details and patterns. Keeping the input size comparatively small and fixed, however, reduces complexity and speeds up training in the computationally restricted environment at hand, potentially at the cost of predictive accuracy though.

The base model is fine-tuned during training, using a simple classification head (compared to MEWC), and consisting of a GlobalAveragePooling2D layer, a Dropout layer and a Softmax activation layer. Learning rate scheduling with exponential decay is used along with an AdamW optimizer, and Categorical (Focal) Cross-Entropy loss. TensorFlow [19], Keras [20], and Keras Images Models (KIMM) [21] are used for model setup and training.

4. Experimental results

Pre-processed images are randomly sampled per class for training, validation (during training), and subsequent testing, respectively, as follows: 2,000, 250, and 250. For this work, these figures seem to approximately constitute the maximum number of images that can be handled within GPU capacity on Kaggle. The additional test

set with pre-processed tiger images from LILA BC contains 296 images. All experiments run for 50 epochs using a single NVIDIA Tesla P100 GPU while early stopping triggers training to end earlier if validation accuracy does not improve for five consecutive epochs (to enable stable training and reduce the risk of overfitting). The considered comparative metrics include accuracy, precision, recall, F1-score, model size, and training time. All experiments are logged using Weights & Biases [22], including details on the respective experiment run. See Table 3 in the Appendix for a summary of the test set performance for all experiments.

The first round of experiments establishes a baseline, training B0, B2, S, M, L, and XL variants of EfficientNetV2. The training time increases with model size as does the predictive performance. Training time ranges from 16 minutes for B0 up to 162 minutes for XL. F1-score ranges from 0.8332 for B0 up to 0.9033 for XL. There appears to be no large discrepancy between validation and test set metrics, and validation loss tends to drop rapidly before plateauing, leaving a relatively minor gap to training loss at the end of training. The variants S and M are selected for the next round of experiments as they strike a balance between predictive performance, model size, and training time.

The second round of experiments shows that augmentations (e.g., horizontal flip, rotation, zoom, contrast, brightness, saturation, and hue) yield only marginal performance improvements (F1-score of 0.8796 and 0.8975 for S and M, respectively) while significantly increasing training time (e.g., from 57 minutes to 163 minutes for M which constitutes an increase by almost 3x while F1-score barely changes). This hints at the fact that the current setup of training images and model design might be near its performance ceiling. Other augmentation techniques, such as MixUp [23], could be tested in a next iteration to potentially boost predictive performance.

The third round of experiments shows that using label smoothing (0.1) slightly improves performance (F1-score of 0.8871 and 0.8986 for S and M, respectively) while not affecting training time dramatically. Label smoothing tends to reduce overconfidence and make the model more robust. Using label smoothing with non-focal Categorical Cross-Entropy loss is easier to implement, and this switch is also justified because the dataset at hand is balanced.

It is worth noting that the test set performance for the tiger class is inflated due to the unrealistic high quality of the images from the tiger substitute dataset used for training and evaluation. Therefore, testing the model on the withheld tiger camera trap images from LILA BC hints at what could be expected as a more realistic predictive performance when the model is applied to actual camera trap images of tigers. In this case, both the S and M variants in the third round of experiments achieve an accuracy of 0.8716 on the tiger class alone, a slight increase for the M variant but a more significant increase for the S variant compared to the previous rounds of experiments.

Finally, EfficientNetV2M without augmentations but with non-focal Categorical Cross-Entropy loss and label smoothing is selected for deployment due to its superior predictive performance, moderate model size, and acceptable training time. The final model is openly accessible on HuggingFace [24] and is deployed in AddaxAI (both is optional for the user).

5. Limitations and challenges

While the initial results of TeraiNet appear promising there remain several limitations and challenges. A common notion to several of them is that an evaluation on local camera trap images of the actual species of interest would prove informative of the model’s ability to generalise to the target region and reliably classify the species of interest correctly. The request for an evaluation, using such data, conducted by WWF Nepal and the Nepal Tiger Trust is in progress (ongoing at the time of writing). Using non-local images and substitute species, however, is rather a limitation of TeraiNet than of the blueprint itself. Key limitations of the blueprint are instead data quality and free storage and compute, and the ensuing restrictions on the number of classes and images, model size, and training time.

5.1. Local camera trap images and substitute species

No local camera trap images from Nepal are openly available on LILA BC or at other sources (e.g., due to restrictions on data sharing by the Nepalese government). In addition, for the development of TeraiNet, some species are substituted by similar species while some classes are enriched with additional similar species since only a limited number of (non-local) images are openly available. Naturally, this strategy constitutes a significant compromise because local images of the actual species of interest are essential for ecological validity and tend to improve predictive performance as well. Therefore, a proper evaluation of TeraiNet’s capacity to generalise to the target region and actual species should be conducted to investigate how this compromise works in practice.

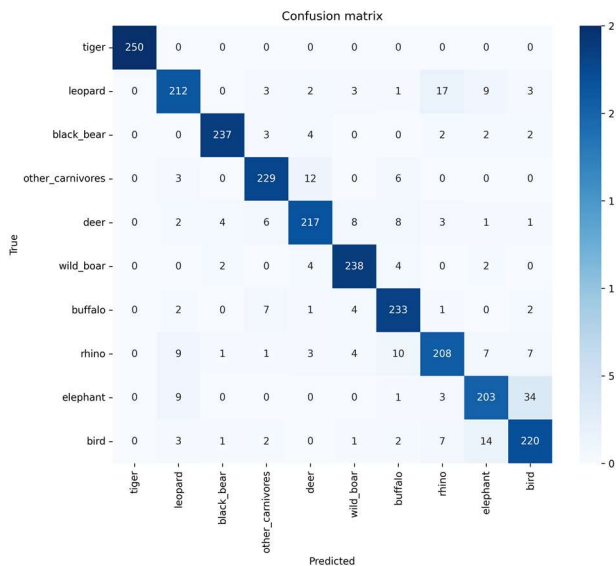


Fig. 2. Confusion matrix of the predictions on the test set by the final model

5.2. Data quality

Confusion between certain species becomes evident during the evaluation of TeraiNet (see Figure 2). In particular, the final model tends to confuse *leopard* and

rhinoceros, *other carnivores* and *deer*, *rhinoceros* and *buffalo*, *rhinoceros* and *bird*, and *elephant* and *bird*. This is partly due to the questionable quality of certain images used for training. There could have been errors during pre-processing where animals are generically detected and cropped. But there could also be cases where the original label is incorrect or several different species are present in a single image (e.g., bird sitting on elephant), leading to partly wrong labels. There are detections available for all the images on LILA BC, including quality checks regarding multiple species in a single image. These cleaned detections could be integrated in a next iteration to somewhat alleviate this issue as they are likely to be of higher quality compared to the output of the detection step in the blueprint’s current setup (and this would further reduce the blueprints complexity as well). Additional measures could be tested to further optimise the quality of the images during sampling from LILA BC (e.g., dataset and timestamp filters).

5.3. Free storage and compute

The free versions of Drive and Colab can be accessed by any user with a respective account and an Internet connection. But they are limited in storage and compute which requires inconvenient workarounds when dealing with large amounts of data or restrict the amount of data altogether. Same applies to Kaggle regarding access to free GPU compute. It can be easily accessed but especially the limited GPU capacity constrains model size and the number of images that can be used for training. Subsequently this also restricts the number of species and images per species, currently forcing the user to decide which species to include and which to exclude. More efficient approaches to optimally allocate limited storage and compute resources could be tested in a next iteration to allow for the integration of more images and species per model.

6. Conclusion

This work focuses on the initial proposition of an open-source blueprint for the fast and flexible development of local species classification models, using openly available data, open tools, and freely accessible storage and compute resources. Its ambition is to enable users to create their own local species classifiers with less hassle and cost.

The development and deployment of TeraiNet, addressing Bengal tiger conservation in the Terai region in Nepal, demonstrates how the blueprint can be put into practice. While the first results appear promising, limitations and challenges become evident, most prominently the lack of local camera trap images and of images for certain species as well as storage and computational restrictions. In its current form, the blueprint tends to be most effective in cases where enough of the right images are openly available while the set of species is limited (e.g., by choice or through restricting the target region geographically). These constraints strengthen ecological validity and operations within computational boundaries of free environments while adding value for conservation through ML.

Further research could dive deeper into individual aspects and challenges of this work, such as using non-local data, species substitution, and efficiently operating in computationally restricted environments.

References

- [1] Gewin, V.: ‘AI carves out a niche in ecology and conservation research’, *Nat. Methods*, 2025, 22, pp. 895–900
- [2] Beery, S., Morris, D., Yang, S.: ‘Efficient pipeline for camera trap image processing’. *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2019, pp. 1732–1740
- [3] van Lunteren, P.: ‘AddaxAI: A no-code platform to train and deploy custom YOLOv5 object detection models’, *J. Open Source Softw.*, 2023, 8, (88), 5581
- [4] ‘DeepFaune’, <https://www.deepfaune.cnrs.fr/>, accessed 26 June 2025
- [5] Beery, S., Morris, D., Yang, S.: ‘Efficient pipeline for camera trap image review’, *arXiv preprint arXiv:1907.06772*, 2019
- [6] Brook, B.W., Buettel, J.C., van Lunteren, P., Rajmohan, P.P., Aandahl, R.Z.: ‘MEWC: A user-friendly AI workflow for customised wildlife-image classification’, *EcoEvoRxiv*, 2025, ver.3, peer-reviewed and recommended by PCI Ecology
- [7] ‘Wildlife Insights’, <https://wildlifeinsights.org/>, accessed 15 June 2025
- [8] ‘Zamba’, <https://www.zambacloud.com/>, accessed 26 June 2025
- [9] ‘LILA BC’, <https://lila.science/>, accessed 26 June 2025
- [10] ‘Google Drive’, <https://drive.google.com/>, accessed 26 June 2025
- [11] ‘Google Colab’, <https://colab.research.google.com/>, accessed 26 June 2025
- [12] ‘Kaggle’, <https://www.kaggle.com/>, accessed 26 June 2025
- [13] Gadot, T., Istrate, S., Kim, H., et al.: ‘To crop or not to crop: Comparing whole-image and cropped classification on a large dataset of camera trap images’, *IET Comput. Vis.*, 2024, 18, (8), pp. 1193–1208
- [14] ‘TeraiNet GitHub repository’, <https://github.com/alexvmt/TeraiNet>, accessed 26 June 2025
- [15] ‘Terai Arc Landscape – WWF Nepal’, https://www.wwfnepal.org/our_working_areas/tal2/, accessed 26 June 2025
- [16] ‘UN Decade on Ecosystem Restoration: Terai Arc Landscape’, <https://www.decadeonrestoration.org/terai-arc-landscape>, accessed 26 June 2025
- [17] Li, S., Li, J., Tang, H., Qian, R., Lin, W.: ‘ATRW: A benchmark for Amur tiger re-identification in the wild’. *Proc. 28th ACM Int. Conf. Multimedia (MM '20)*, Seattle, WA, USA, Oct. 2020, pp. 2590–2598
- [18] Tan, M., Le, Q.: ‘EfficientNetV2: Smaller models and faster training’. *Proc. 38th Int. Conf. Machine Learning (ICML)*, 2021, PMLR 139, pp. 10096–10106
- [19] Abadi, M., Barham, P., Chen, J., et al.: ‘TensorFlow: A system for large-scale machine learning’. *Proc. 12th USENIX Conf. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283
- [20] Chollet, F.: ‘Keras: Deep learning library for Theano and TensorFlow’, <https://github.com/fchollet/keras>, accessed 26 June 2025
- [21] Chiu, H.: ‘Keras Image Models’, GitHub repository, <https://github.com/james7777778/kimm>, accessed 26 June 2025
- [22] ‘TeraiNet – Weights & Biases Project’, <https://wandb.ai/alexvmt/TeraiNet/table>, accessed 26 June 2025
- [23] Zhang, H., Cisse, M., Dauphin, Y.N., Lopez-Paz, D.: ‘mixup: Beyond Empirical Risk Minimization’, *arXiv preprint arXiv:1710.09412*, 2018
- [24] ‘TeraiNet – Hugging Face Repository’, <https://huggingface.co/alexvmt/TeraiNet/tree/main>, accessed 26 June 2025

Appendix

Table 2 Number of images per class for selected development pipeline steps

Class	Sampled	Cropped	Final Train/ Val/Test
Tiger	4,736	7,861	2,500
Leopard	2,991	2,648	2,500
Black bear	3,100	2,639	2,500
Other carnivores	3,101	2,775	2,500
Deer	3,100	3,506	2,500
Wild boar	3,100	3,260	2,500
Buffalo	3,100	5,811	2,500
Rhino	3,100	5,778	2,500
Elephant	3,100	5,317	2,500
Bird	3,100	3,115	2,500

Table 3 Summary of test set performance (aug = augmentations; ls = label smoothing)

Experiment	F1-score (all classes)	Accuracy (withheld tiger images)
B0	0.8332	0.8108
B2	0.8527	0.8277
S	0.8768	0.8412
M	0.8927	0.8649
L	0.9071	0.8581
XL	0.9033	0.8682
S + aug	0.8796	0.8277
M + aug	0.8975	0.8615
S + ls	0.8871	0.8716
M + ls	0.8986	0.8716